

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly integrated with graph-structured data for tasks such as node classification, extending a setting historically dominated by Graph Neural Networks (GNNs). Although this integration enables models to exploit both relational and textual signals, the adversarial robustness of graph-aware LLMs has not been systematically characterized. This paper presents the first systematic study of adversarial attacks and defenses for graph-aware LLMs by evaluating representative poisoning (training-time) and evasion (test-time) graph attacks on two projector-based models, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024). The study further reveals a graph-aware attack surface specific to LLAGA: malicious nodes can be injected into placeholder positions in the node sequence template, causing substantial performance degradation. Across the evaluated datasets and attack settings, the results establish that graph encoding design is a central determinant of robustness: (1) the node sequence template in LLAGA amplifies sensitivity to structural manipulation; (2) the GNN encoder used in GRAPH-PROMPTER provides comparatively stronger resistance to structural perturbations; and (3) both projector paradigms remain highly exposed to imperceptible perturbations of textual node features. Finally, we introduce GALGUARD, an end-to-end defense framework that combines LLM-based feature correction with adapted GNN defenses for structural perturbations¹

1 Introduction

Graphs encode entities and their relations as nodes and edges, providing a foundational abstraction for complex relational systems in social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018) have consequently become a standard modeling paradigm for graph-structured tasks such as node classification and link prediction. The rapid development of large language models (LLMs), including GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), creates a complementary opportunity: models trained on large-scale

text can be adapted to exploit the textual attributes and relational organization that co-occur in many graph datasets. This integration, referred to as *graph-aware LLMs* (Xie et al. 2023), has become an active research direction in which LLMs are applied to tasks previously handled primarily by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Recent results show that graph-aware LLMs can outperform GNNs on several graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), positioning them as substantive alternatives rather than merely auxiliary language components.

This growing role makes robustness a central requirement, yet the adversarial vulnerability of graph-aware LLMs remains largely unexamined. Existing attacks on GNNs, including poisoning (training-time) and evasion (test-time) attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), commonly assume a black-box adversary with no access to internal model architecture or parameters. Such attacks compromise *model integrity* by subtly modifying graph structure or node features so that predictive performance degrades. The graph adversarial literature has primarily emphasized structural perturbations; graph-aware LLMs, however, add a textual interface in which visually imperceptible changes to node attributes can alter the token sequence processed by the LLM while remaining benign to human readers.

These observations motivate the central question of this paper: do *adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm*, and what additional vulnerabilities arise from the mechanisms used to encode graphs for LLMs? The answer depends on the interface through which graph information enters the language model. Textual node features are naturally compatible with LLM inputs, whereas graph structure must be transformed through an encoding mechanism that preserves relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore organize graph-aware LLM methods into two encoding classes: approaches that represent graphs as *textual descriptions*, and approaches that use *learned projectors* to translate graph information into representations consumed by a frozen LLM. Because textual-description methods inherit well-studied text attack surfaces

¹Our code will be released upon acceptance.

and often face scalability limitations, our empirical analysis focuses on learned projector methods, represented by two computationally efficient models, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), both of which retain frozen pre-trained LLM backbones.

Our study establishes that graph encoding design is a decisive factor in adversarial robustness for graph-aware LLMs. In particular, we identify a new attack surface in LLAGA: by manipulating the node sequence template so that malicious nodes occupy placeholder positions, an attacker can substantially degrade performance. We further show that feature perturbations, which are often secondary in the GNN setting, become a major vulnerability once textual node attributes are processed by LLMs. To complement the attack analysis, we propose GALGUARD, an end-to-end graph-aware LLM defense that integrates LLM-based feature correction with adapted GNN-based structural defenses. To our knowledge, this is the first systematic investigation of adversarial attacks and defenses for graph-aware LLMs. The paper makes five connected contributions:

- We systematically evaluate the transferability of representative GNN attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to graph-aware LLMs under poisoning and evasion settings.
- We provide a graph encoding-based taxonomy of graph-aware LLMs that clarifies how different methods expose graph structure to a frozen LLM and motivates the focus on projector-based models.
- We identify and evaluate a LLAGA-specific attack surface in which malicious nodes are injected as placeholders in the node sequence template.
- We characterize imperceptible feature perturbation attacks on textual node attributes and show that they constitute a central vulnerability for graph-aware LLMs.
- We introduce GALGUARD, an end-to-end defense strategy that combines LLM-based feature correction with adapted GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs fall into two principal categories: *textual descriptions of graphs*, which serialize graph information as natural language, and *learned graph projectors*, which map graph data into embeddings or structured token-level representations that an LLM can process. This distinction is central to the robustness question studied here because the encoding mechanism determines how structural and textual perturbations reach the LLM input. Figure 5 illustrates both categories; Appendix A.1 provides the full taxonomy, and Appendix A.2 reviews related work on GNN attacks.

Graph Representation as a Textual Description. Textual-description approaches convert graph data into natural language so that LLMs can consume graph information through prompts and instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al.

2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) associate molecular graphs with textual descriptions. Related work also studies how graph structure can be incorporated into prompts; for example, Huang et al. (2023a) shows that LLMs often process graph prompts as contextual paragraphs rather than explicit graph objects, making prompt design consequential (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Additional methods use LLMs to enhance node features or graph structures, act as predictors, or serve as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) combine LLMs and GNNs for textual graphs, and Duan et al. (2023) introduce an effective textual graph learning approach. These methods provide a direct route for applying LLMs to graph tasks, but they can struggle to preserve structural detail, particularly in large graphs, and they remain sensitive to prompt engineering choices.

Graph Representation as a Learned Projector. Learned-projector methods address these limitations by transforming graph data into embeddings, graph vectors, or token-like structures that can be aligned with the LLM input space. Representative approaches include LLAGA, which constructs node-level templates and applies a linear projector (Chen et al. 2024), and methods that incorporate GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Other variants use graph neural prompts to encode knowledge graphs (Tian et al. 2024), prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or graph transformers that compress graph information into fixed-length prefixes (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). Because these methods make the graph-to-LLM interface explicit, they provide a precise setting for analyzing how encoding choices mediate adversarial perturbations.

This Work: We focus on projector methods built on *frozen pre-trained LLMs* because they are computationally efficient and avoid the substantial cost of retraining the language model (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Our empirical study uses two representative designs: LLAGA (Chen et al. 2024), which relies on node-level templates and a linear projector, and GRAPHPROMPTER (Liu et al. 2024), which combines a GNN encoder with a linear projector. This choice targets the graph-specific robustness mechanisms introduced by learned projectors; textual-description methods are outside the empirical scope because their scalability limitations and conventional text-prompt attack surfaces are less aligned with the structural vulnerabilities examined in this paper.

3 Threat Model

We define the threat model by extending established graph adversarial settings to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The setting captures realistic limited-control scenarios. In a social network, for example, an attacker may control only a small number of bot accounts

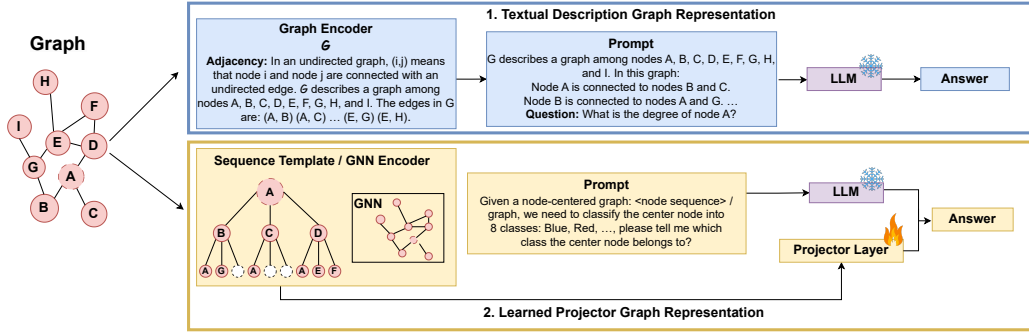


Figure 1: **Graph encoding-based adaptations of LLMs.** Textual-description methods express graph information as natural language (top), whereas learned-projector methods encode graphs through node templates that convert neighborhoods into sequences or through GNN encoders (bottom). In both projector cases, the LLM remains frozen and only the graph-to-token projector is trained.

yet still manipulate local graph structure or textual attributes to evade detection or influence the classification of legitimate users. In a Wikipedia-like graph, sparsely connected hoax articles may be strategically linked to authentic articles to induce misclassification of genuine content. These threats are especially consequential as graph-aware LLMs are considered for recommendation, social media analysis, healthcare, and other domains in which they may supplement or replace GNN-based pipelines. Following prior work, we specify the threat model through model access, adversarial capabilities, attack strategies, and the attacker’s objective.

Model Access. The attacker operates in a black-box setting and has no knowledge of the target model’s architecture or parameters. The attacker can influence the model only through the training data or through test-time inputs.

Adversarial Capabilities. Consistent with poisoning and evasion attacks, respectively, the attacker has access to either the training data or the inference-time data of the target model. Under a predefined perturbation budget and unnoticeability constraint, the attacker may modify existing data or inject additional data.

Attack Strategies. The attacker uses two perturbation channels: feature manipulation, which alters node attributes, and structural manipulation, which adds or removes edges.

Attacker’s Goal. The attacker seeks to compromise model integrity by degrading overall classification performance, operationalized as an increased misclassification rate.

4 Experimental Setup

Dataset. We evaluate on three widely used text-attributed graph benchmarks, Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016). These datasets span small to medium graph scales and differ in sparsity, making them standard evaluation settings for the graph-aware LLM models studied here (Chen et al. 2023a, 2024; Liu et al. 2024). To assess behavior on a larger graph, we additionally report an ArXiv experiment (Hu et al. 2020) in Appendix A.5. We use the standard train/validation/test splits of 6:2:2 for Cora, Citeseer, and PubMed, and 5:2:3 for ArXiv, following (Chen et al. 2024). Table 1 summarizes the dataset statistics used in

the main experiments.

Table 1: Statistics of the text-attributed graph datasets used in the main evaluation.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We use *node classification* as the primary evaluation task because it is a standard benchmark for graph machine learning and the canonical setting for the adversarial attacks considered in this work (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, defined as the proportion of correctly classified nodes. Each experiment is repeated three times, and all tables report the mean and standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We select graph-aware LLMs and attack methods to cover representative projector designs and established graph adversarial mechanisms.

Graph-Aware LLM Methods. We evaluate two representative graph-aware LLMs: LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024). Both methods use a *frozen pre-trained LLM*, which makes them computationally efficient and transferable across graph tasks and LLM architectures. LLAGA constructs node-level templates and maps them through a linear projector, whereas GRAPH-PROMPTER uses a GNN-based encoder followed by a linear projector. In both models, the projector is a 2-layer multi-layer perceptron (MLP). The base LLMs for LLAGA and GRAPH-PROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We employ the poisoning

Table 2: Accuracy of LLAGA and GRAPH PROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed. Percentages in parentheses denote the drop from clean accuracy.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

and evasion attack from two representative adversarial attack methods NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019) to conduct the robustness evaluation. NETTACK is a targeted attack that identifies minimal perturbations sufficient to misclassify selected nodes, whereas METAATTACK performs an untargeted attack by optimizing graph structure as a hyperparameter. For NETTACK, we follow (Li et al. 2023; Zügner and Günnemann 2019): each node is targeted sequentially and the resulting outcomes are aggregated, yielding an untargeted evaluation protocol. Across experiments, we perturb only 10% of edges to preserve the unnoticeability constraint used in prior work (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Appendix A.2 provides additional attack details.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We evaluate poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to graph-aware LLMs. In poisoning attacks on LLAGA, the perturbed graph is used to generate node-level sequences before retraining the projector. In poisoning attacks on GRAPH PROMPTER, the perturbed graph is re-encoded with a GAT and the projector is retrained. In evasion attacks, both models are trained on clean data, and perturbations are introduced only at inference time.

Results. Table 2 shows three consistent empirical patterns. First, poisoning attacks have a limited effect in this evaluation, with maximum degradation of 11% across the reported attack settings, whereas evasion attacks are substantially more damaging, reaching a 51% drop. This contrast indicates that the trained projector and the frozen LLM context provide more resilience when perturbations are present during training than when they appear only at inference. Second, METAATTACK generally produces stronger degradation than NETTACK under both poisoning and evasion, consistent with its ability to optimize graph edges as hyperparameters and identify small but effective structural changes. Third, denser graphs are more vulnerable than sparser graphs in the reported settings: for example, LLAGA drops by 51% on the denser Cora graph under METAATTACK evasion, compared with 19% on the sparser PubMed graph.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

The comparison in Table 2 indicates that LLAGA is more susceptible than GRAPH PROMPTER because its representation is more directly tied to perturbed graph structure. LLAGA incorporates structure by constructing a fixed-length node-level sequence for each node and by computing a graph Laplacian embedding. Each sequence position corresponds to a relative structural position in the original graph; consequently, when the graph is perturbed, the node embeddings delivered to the LLM are also perturbed. GRAPH PROMPTER, by contrast, preserves textual node embeddings under structural perturbation because the attack affects the GNN component rather than the text representation. This separation allows unperturbed textual features to compensate for corrupted structural information, yielding stronger robustness in the evaluated settings.

6 Node Sequence Template Injection Attacks

We identify a new attack surface in LLAGA by exploiting the neighborhood detail template used to construct node sequences. Figure 2 illustrates the mechanism, and Appendix A.4 provides the algorithm. To obtain the node sequence for a node u , LLAGA constructs a *fixed-shape* computational tree whose children are sampled 1-hop neighbors; the procedure is then repeated at subsequent tree levels. When a node has too few neighbors, placeholders are inserted to preserve the fixed width, such as $k = 10$ children in the original LLAGA design (see "Clean" from Figure 2 and lines 1-14 in the algorithm). This placeholder mechanism exposes a graph-specific injection surface.

We instantiate this vulnerability through three injection strategies. *Non-Adjacent Injection (NI)* inserts nodes that are outside the 2-hop neighborhood of a target node u (see "Attacked" from Figure 2 and lines 21-27). *Supernode Injection (SI)* inserts a high-degree node as a placeholder while keeping it outside the target node’s 2-hop neighborhood. *Multiple Supernode Injection (MSI)* extends SI by injecting multiple distinct supernodes as placeholders (lines 35-43). In each case, uninformative or adversarially selected nodes enter the node sequence and corrupt the target representation, which in turn degrades the projector’s training and inference behavior. **Results.** As shown in Table 3, LLAGA remains relatively resilient to poisoning on Cora and Citeseer but is sharply

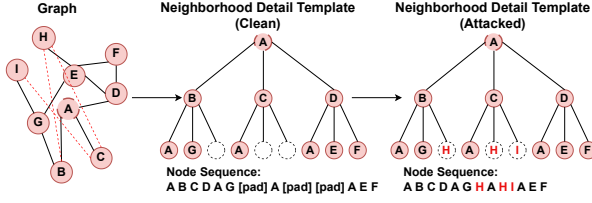


Figure 2: Illustration of node sequence template injection attacks. The example shows Non-adjacent Injection (NI), where red dashed lines denote attacker-induced edge perturbations. In Supernode Injection (SI) and Multiple Supernode Injection (MSI), the injected supernode or supernodes occupy placeholder positions. Here, $k = 3$.

Table 3: Accuracy of LLAGA under node sequence template injection attacks. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively. Clean reports performance without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	——	0.89 ± 0.07	——
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	——	0.71 ± 0.04	——
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	——	0.90 ± 0.04	——
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

degraded under evasion. On Cora, accuracy decreases to 0.42, 0.36, and 0.30 under *NI*, *SI*, and *MSI*, respectively; on Citeseer, the corresponding accuracies are 0.36, 0.30, and 0.24.

On the larger PubMed dataset, LLAGA exhibits stronger resilience to all injection attacks, with evasion accuracies of 0.85, 0.82, and 0.76. This pattern is consistent with the structural scale of the datasets: in smaller and more tightly connected graphs such as Cora and Citeseer, a localized injection can more strongly disrupt neighborhood reasoning, whereas in the larger PubMed graph the same perturbation is comparatively diluted.

Relative to the traditional graph adversarial baselines, the node injection attacks expose a stronger weakness in LLAGA on Cora and Citeseer. For example, *MSI* yields evasion accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, whereas METAATTACK yields 0.44, 0.50, and 0.73 under the same datasets (Table 2).

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks primarily focus on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019), with the exception of NETTACK (Zügner, Akbarnejad, and

Günnemann 2018), which is not transferable to the textual domain. Given that structural attacks are only moderately successful on robust models like GRAPH PROMPTER, and that textual features have proven to be important for model robustness, we explore unnoticeable feature perturbation attacks to degrade model performance.

We adopt the imperceptible attack strategies proposed by (Boucher et al. 2022) but with a key modification: we use a task-agnostic translation objective (English to French) to generate perturbations. The objective is to modify the input text such that the changes are visually identical but lead to an incorrect translation, thereby corrupting the encoded semantics. The intuition is that if the translation for a given text is wrong, the encoded semantics would also be wrong. We employ two imperceptible attacks: *Homoglyphs*, which replaces characters with visually similar ones and *Reorderings*, which use direction control characters to alter the rendering sequence. Examples of Homoglyphs and Reorderings are in Appendix A.6.

Following (Boucher et al. 2022), we use differential evolution (Storn and Price 1997) to optimize these perturbations, maximizing the Levenshtein distance between the model’s output on the perturbed and unperturbed text. The perturbation budget is set to 10% of the average length of each textual feature.

Results. As shown in Table 4, Reordering attacks are generally more potent than Homoglyph attacks across all models and datasets. On Cora and CiteSeer, LLAGA shows vulnerability to both attack types, with poisoning attacks causing accuracy drops of 27% and 23% for Homoglyph and a more significant 35% and 25% for Reordering, respectively. In both cases, GRAPH PROMPTER proves more resilient. The PubMed dataset exhibits similar trends, though the overall impact is less pronounced than on the smaller datasets.

A comparison with structural perturbations reveals a fundamental shift in vulnerabilities, as shown in Figure 3. Feature perturbation attacks (specifically Reordering) outperform structural attacks (METAATTACK) by a large margin on both LLAGA and GRAPH PROMPTER. This suggests that graph-aware LLMs place a greater emphasis on input features than on graph structure, making them more vulnerable to feature perturbations. We further explore this by combining both attack types in a unified setting in the next section.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that combines both feature and structural perturbations to create a more imperceptible and effective adversarial strategy. We hypothesize that combining these attack types will more significantly degrade model accuracy than either method alone, while maintaining imperceptible alterations to the graph structure and features.

For the structural perturbation, we use METAATTACK, which modifies graph structure by subtly adjusting edges. For the feature component, we use a combination of Homoglyph and Reordering attacks, as described in previous section. Together, these perturbations create a highly imperceptible attack that simultaneously targets the relational and textual

Table 4: Performance of feature perturbation attacks, with percentage drop from clean accuracy in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 ± 0.05 (27%)	0.52 ± 0.04 (13%)	0.41 ± 0.04 (54%)	0.38 ± 0.03 (37%)
	Reordering	0.58 ± 0.06 (35%)	0.45 ± 0.05 (25%)	0.24 ± 0.05 (73%)	0.20 ± 0.03 (67%)
Citeseer	Homoglyph	0.54 ± 0.04 (23%)	0.65 ± 0.04 (7%)	0.32 ± 0.03 (55%)	0.52 ± 0.05 (26%)
	Reordering	0.46 ± 0.03 (35%)	0.39 ± 0.02 (44%)	0.20 ± 0.05 (72%)	0.27 ± 0.02 (61%)
PubMed	Homoglyph	0.72 ± 0.04 (20%)	0.83 ± 0.04 (8%)	0.36 ± 0.06 (60%)	0.55 ± 0.03 (39%)
	Reordering	0.67 ± 0.05 (26%)	0.80 ± 0.03 (11%)	0.28 ± 0.04 (69%)	0.42 ± 0.04 (53%)

Table 5: Model accuracy under the unified imperceptible attack. Percentage drop from clean accuracy in parentheses.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 ± 0.05 (61%)	0.28 ± 0.08 (53%)
	Evasion	0.14 ± 0.06 (84%)	0.11 ± 0.05 (82%)
Citeseer	Poisoning	0.26 ± 0.04 (63%)	0.22 ± 0.03 (69%)
	Evasion	0.10 ± 0.09 (86%)	0.13 ± 0.07 (81%)
PubMed	Poisoning	0.39 ± 0.07 (57%)	0.32 ± 0.04 (64%)
	Evasion	0.12 ± 0.05 (87%)	0.19 ± 0.04 (79%)

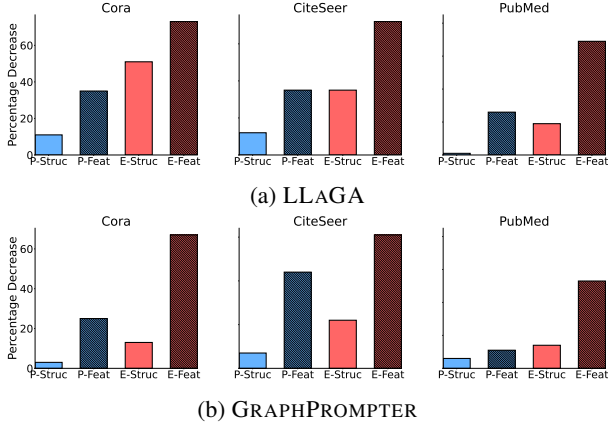


Figure 3: Performance of structure and feature-based adversarial attacks on graph-aware LLMs. The attack success rate here is measured by the percentage decrease (y-axis) to the original accuracy. P-Struc, P-Feat, E-Struc, E-Feat refer to the poisoning and evasion attacks based on structural perturbation and feature perturbation.

information on which graph-aware LLMs rely.

Results. As shown in Table 5, the unified attack approach, which jointly applies structural and feature perturbations, is significantly more effective than either perturbation method alone in degrading model performance across various datasets. This analysis offers a key insight into the architecture of graph-aware LLMs: unlike traditional GNNs, which are primarily impacted by structural perturbations, these models exhibit substantial vulnerability to feature-level attacks. The effectiveness of the unified attack indicates that these models rely on the consistency of both graph structure and node features, making them highly susceptible to such combined, inconspicuous modifications.

9 Ablation Study

9.1 Varying Feature Perturbation

To better understand the relationship between perturbation level and attack success, we study the impact of varying the percentage of perturbed features. Our analysis of the Reordering attack, which exhibits similar patterns to the Homoglyph

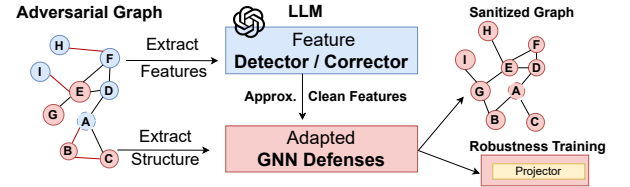


Figure 4: **Overview of GALGUARD defense.** Perturbed node features and edges are colored in blue and red, respectively. The LLM identifies perturbations in node features extracted from an adversarial graph and generates a corrected version. The refined features, together with the perturbed edges, are then passed through existing GNN defenses, which can either be used to obtain a sanitized graph or adapted for projectors' robustness training.

attack, reveals a consistent trend as the perturbation percentage varies from 1% to 10%. For all models and datasets (Cora, Citeseer, and PubMed), reducing the percentage of perturbed features leads to a less successful attack, as expected. Notably, as the perturbation level decreases, the success rates of poisoning and evasion attacks tend to converge. This is because with fewer perturbations, the adversarial signal becomes too weak to significantly impact the model, regardless of whether it was introduced during the training (poisoning) or inference (evasion) phase. A detailed plot illustrating these trends is in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We evaluate whether the foundation LLM influences adversarial robustness in graph-aware LLMs. LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPHPROMPTER uses Llama2-7B (Touvron et al. 2023). We replace LLAGA's backbone with Llama2-7B and compare performance under poisoning and evasion attacks on all datasets. Both models achieve similar accuracy in clean settings (e.g. Vicuna: 0.89 ± 0.07 , Llama2: 0.90 ± 0.03 on Cora) and exhibit comparable vulnerability under attacks. The trend holds when swapping LLMs in GRAPHPROMPTER. These results suggest that the choice of foundation LLM has minimal impact on adversarial robustness. The full results are in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses against adversarial attacks are ineffective for graph-aware LLMs due to their dual vulnerability to structural and feature perturbations. Many defenses rely on node feature similarity to prune or reweight edges (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), but they assume access to clean features. When features are perturbed, these methods fail (see Figure 3 in the Appendix). Similarly, defenses that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021) and even OCR-based sanitization (Boucher et al. 2022) break down under coordinated attacks. This dual vulnerability necessitates a new, integrated defense.

To address this, we propose graph-aware LLM defense (GALGUARD), an end-to-end defense strategy combining two complementary approaches: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As shown in Figure 4, an LLM serves as an adaptive mechanism to analyze textual node features for anomalous patterns. It provides approximate corrections to restore feature integrity, thereby mitigating the effect of feature-level perturbations. This LLM-based correction is a practical solution for feature vulnerabilities. The specific instructions and prompts are in Appendix A.6. For our experiments, we employ GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. To enhance structural robustness, we employ a two-step approach. First, we adopt a *graph purification heuristic* (Wu et al. 2019) as a preprocessing step. For each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and its neighbors’ embeddings $\phi(x_u)$ and remove edges with low similarity. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). We directly apply GNNGuard’s principles to the message-passing layers of GRAPHPROMPTER. For LLAGA, we propose a novel adaptation: we incorporate the purification step into the construction of the computational tree for its node sequence. Furthermore, we introduce a learnable *global structural context embedding* (M_{global}) as a learnable parameter in LLAGA’s projector. By concatenating M_{global} with the purified node sequence embedding and jointly training them, the model learns to operate robustly on potentially perturbed graph inputs. The plug-and-play design of GALGUARD’s allows other defenses, e.g. Pro-GNN (Jin et al. 2020), to be used instead of GNNGuard.

10.1 Results

Table 6 summarizes the performance, measured in accuracy, of models under METAATTACK, as well as our defense approaches: GALGUARD_p, which combines a feature corrector with a graph purification module, and the full GALGUARD, which integrates the feature corrector, the graph purification module, and GNNGuard (or its adaptation). Across all evaluated datasets (Cora, CiteSeer, and PubMed) and for both poisoning and evasion attack types, our defenses consistently mitigate the impact of METAATTACK. Specifically, GALGUARD_p proves to be an effective foundational step,

demonstrating that addressing feature perturbations with the LLM corrector and applying initial graph purification significantly improve robustness. Notably, the full GALGUARD consistently yields superior improvements in model robustness compared to GALGUARD_p, highlighting the synergistic effect of integrating the GNN-inspired structural defense.

Table 6: Performance of METAATTACK and the proposed defense GALGUARD under poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification

Data Method		LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

In this paper, we take a substantial first step in uncovering the vulnerabilities of graph-aware LLMs to adversarial attacks, a largely unexplored area. Our work reveals that the methods used for integrating graph data are a critical source of vulnerability. We discovered new attack surfaces, demonstrating that design choices like the node sequence template in LLAGA significantly increase a model’s susceptibility. Furthermore, our findings show that feature perturbation attacks are highly effective (unlike traditional GNNs), often outperforming traditional structural attacks, and that a unified attack combining both perturbations can render graph-aware LLMs near-useless.

To address this, we propose an end-to-end defense, GALGUARD, which integrates an LLM-based feature corrector with adapted GNN-based structural defenses to provide robust protection. As these models become more prevalent and has potentials in applications spanning social networks, healthcare, and finance, ensuring their resilience to adversarial exploitation is of the essence. We hope this work serves as a foundation for further research, inspiring deeper exploration into the vulnerabilities and defenses of graph-aware LLMs and fostering both theoretical insights and practical safeguards for their real-world deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for

- medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.

- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure-text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.

Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.

Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.

Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.

Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.

Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.

Zhang, A.; and Ma, J. 2024. Defensevae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.

Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.

Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.

Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.

Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.

Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.

Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.

Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.

Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.

Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is organized as follows. Appendix A.1 provides a detailed taxonomy of graph-aware LLMs. We discuss related work on attacks against Graph Neural Networks (GNNs) in Appendix A.2. Appendix A.3 gives an overview of the adversarial attacks, NETTACK and METAATTACK, while Appendix A.4 details the algorithm for the node sequence injection attack on LLAGA. We present the results of our ablation studies in Appendix A.5, including the effect of varying the amount of perturbed features and the choice of base LLM on attack performance. Finally, Appendix A.7 shows examples of feature attacks and the template for the LLM corrector, and Appendix A.8 discusses the implications of our research.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has been an emerging area of research, with various approaches exploring how LLMs can be adapted or enhanced for graph-related tasks such as reasoning, representation, and inference. These approaches can be largely divided into two categories based on their methods for encoding graphs for LLMs: *textual descriptions of graphs*, where graph information is encoded as natural language, and *learned graph projector*, where a projector is utilized to encode graph data into embeddings or structured forms that LLMs can directly process. In this section, we describe the methods used in the different encoding approaches for integrating graph data into LLMs. Figure 5 provides the illustration of both approaches and Table 7 provides a summary of the differences between both approaches.

Graph Representation as a Textual Description In this first category, graph data is represented as natural language, allowing LLMs to process graph structures as textual descriptions. This approach has been explored extensively with works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) converting graph data into textual formats, often paired with specific instructions to query LLMs. For example, Tan et al. (2023) introduced a unified framework for attributed graph embedding by generating random walk-based textual sequences from graphs, which are then used to fine-tune LLMs. Similarly, Zhao et al. (2023) focused on graph reasoning within text spaces, translating graph structures into natural language to facilitate LLM interaction. Extending this idea, Wei et al. (2024a) augmented textual representations of graphs with visual representations of the graph, enhancing the reasoning capabilities of LLMs on graph data. In multimodal applications, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions, enabling advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) investigated how incorporating graph structure into LLM prompts can enhance predictive performance. Their findings revealed that LLMs tend to process graph-related prompts as contextual

paragraphs, rather than explicitly recognizing graph structures. This further underscores the importance of prompt design, as seen in studies like (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024), which extended LLMs to heterogeneous and large-scale graph tasks. Complementing these efforts, (He et al. 2024) explored using LLM-generated explanations as features to enhance both the performance and interpretability of LLMs on graphs.

While still representing graphs as textual description, another line of work integrates pretraining strategies with GNNs to enhance the LLMs’ graph reasoning capabilities. For instance, Chen et al. (2023a) proposed two LLM-GNN integration approaches: 1) using LLMs to enhance node features for node classification, and 2) employing LLMs as direct predictors for graph tasks. In line with this, Chandra et al. (2020) utilized LLMs as text encoders and GNNs as graph structure encoders for tasks such as fake news detection in online communities. Similarly, Chen et al. (2023b) explored the possibility of performing node classification without explicit labels by using LLMs to annotate nodes, which are then used in training a GNN model. Likewise, Mavromatis et al. (2023) utilized LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graph that combined LLMs and GNNs through a tunable side structure alongside each layer of the LLM, enhancing both training efficiency and inference speed. Similarly, Duan et al. (2023) introduced SimTeG, a simple yet effective approach that enhances textual graph learning by leveraging LLMs. In their method, the LLM is first fine-tuned on a downstream task, such as node classification. Afterward, node embeddings are generated by extracting the last hidden states from the fine-tuned LLM. Additionally, Zhang (2023) proposed Graph-toolformer, which leverages prompt engineering, powered by ChatGPT, to augment LLMs’ graph reasoning capabilities.

While these approaches make LLMs more accessible for graph-related tasks, they often struggle to fully capture the structural intricacies inherent in graph data. Moreover, the performance of these methods remains heavily dependent on effective prompt engineering, which can be a challenging process, requiring careful design and experimentation to obtain optimal results.

Graph Representation as a Learned Projector To address the challenges posed by textual representation approaches, learned projector methods offer an alternative by transforming graph data into embeddings or graph vectors that LLMs can process more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that retains the general-purpose nature of LLMs while transforming graph structures into graph tokens, a format suitable for LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporated graph structure information into a pre-trained LLM through a tailored disentangled graph neural network layers. Tian et al. (2024) proposed using graph neural prompts to encode knowledge graph, transforming them into graph embeddings that LLMs can leverage during inference. Furthermore, Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) for few-shot

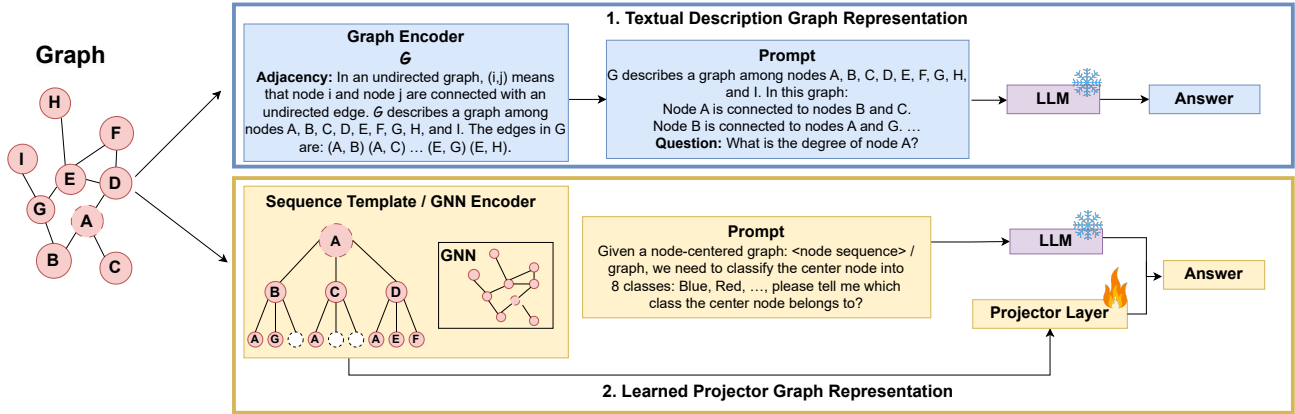


Figure 5: **Graph encoding-based adaptations of LLMs.** In the textual description graph representation, graphs are explained textually (top part). In the learned projector representation, graphs are encoded either by node templates that turns the graph into sequences or by GNNs (bottom part). Note that in both cases, the LLM is frozen. Only the projector that maps the graph into tokens is trained.

Table 7: Comparison of graph encoding-based adaptations of LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

learning on text-attributed graphs. Their method combines a graph adapter (a learnable GNN layer) with task-specific prompts to capture neighborhood information and extract informative node features.

Another promising line of work condenses graph information into fixed-length prefixes using graph transformers, which are attached to each layer of the LLM to enhance its reasoning capabilities (Chai et al. 2023). Similarly, Yang et al. (2021) developed GraphFormers which seamlessly integrates text encoding and graph aggregation in an iterative workflow by nesting GNNs alongside the transformer blocks of a language model. Additionally, Peng et al. (2024) developed ChatGraph, an interactive interface that allows users to interact with their graphs through natural language. Here, graphs are represented as sequential paths and the LLM is fine-tuned on the graph data to support graph analysis.

In this paper, we do not consider projector adaptations that require retraining the LLM, as they significantly increase computational costs without yielding substantial performance gains. Instead, we focus on adaptation methods that utilize **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This approach not only reduces computational costs but also offers greater flexibility in integrating with various LLM models. Additionally, we can take advantage of LLMs’ extensive knowledge and capabilities without the overhead of retraining while still adapting them for specific graph tasks. Throughout this work, we employ two representative projector methods: **LLAGA** (Chen et al. 2024), which utilizes node-level templates and a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which employs a GNN-based projector to encode the graph.

This work. In this work, we do not consider textual description methods, as they often struggle to fully capture the structural intricacies inherent in graph data. These methods are generally limited to simpler tasks, such as describing basic graph properties like node degree, and become infeasible for larger graphs due to scalability issues. In particular, the token limit in prompts restricts the detailed descriptions required for larger graph structures, quickly exceeding allowable token counts. Additionally, from a vulnerability perspective, textual description methods do not introduce any unique attack surface; since the graph is represented as simple text descriptions (e.g., “Node A is connected to Node B”), they are susceptible to traditional text-based adversarial attacks rather than the structural or feature perturbations specific to graph-based models. Therefore, they lack the complexity that would expose them to novel, graph-specific vulnerabilities, making them less relevant for our investigation into adversarial robustness of graph-aware LLMs.

A.2 Attacks on GNN

Several types of attacks exist on GNNs, including membership inference attacks (Olatunji, Nejdli, and Khosla 2021; He et al. 2021; Conti et al. 2022), which aim to determine whether a specific node was part of the model’s training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit the GNN’s output and learned embeddings to infer

sensitive or missing node attributes; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which seek to extract global properties of the entire graph; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which attempt to degrade the model’s performance by introducing subtle perturbations to the graph or node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which aim to reconstruct the underlying graph structure using the learned embeddings or model outputs.

Among these attack types, adversarial attacks are of particular concern due to their direct impact on the model’s performance. In adversarial attacks, small, carefully crafted *perturbations* are introduced into the graph, such as modifying node features or altering edges, with the goal of significantly degrading the GNN’s performance. These attacks can be categorized into two types: **poisoning attacks**, which occur during the *training phase* by manipulating the training data, and **evasion attacks**, which occur at *test time*, where the adversary introduces perturbations to the input graph to mislead the model’s predictions. For instance, Zügner, Akbarnejad, and Günnemann (2018) demonstrated that adversarial perturbations on graph structures or node features can significantly degrade the performance of GNNs. Their method, **NETTACK**, employed a greedy optimization technique to compute the minimal perturbations that would maximize the model’s loss function during training. This approach effectively identifies the smallest changes in the features or graph structure needed to disrupt the model’s predictions. Li et al. (2023) revisited the problem of adversarial attacks on graph data from a data distribution perspective, highlighting how adversarial examples can significantly alter the underlying data distribution and proposed a novel defense mechanism to mitigate such effects. Similarly, Wu et al. (2019) provided a comprehensive analysis of adversarial examples on graph data, offering deep insights into both attacks and defenses. They focused on the vulnerabilities of GNNs to small perturbations and proposed defense techniques designed to detect and counteract adversarial modifications, thus enhancing model robustness. **METAATTACK** (Zügner and Günnemann 2019) performed both poison and evasion attacks on GNNs by treating the graph structure as a hyperparameter that can be optimized. The attack directly modifies the graph structure using gradient descent, formulating the problem as a bi-level optimization task. Their method utilized meta-gradients, which capture how small perturbations in the graph structure affect the attacker’s loss after training, enabling precise updates to the graph that degrade the model’s performance.

In this paper, we leverage **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attack methods, encompassing both poisoning and evasion strategies, to conduct our investigation. Specifically, poisoning attacks target model integrity during training, while evasion attacks compromise performance at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK performs graph structure perturbation by adding and removing edges between nodes, while ensuring that the degree of the perturbed graph is preserved. It uses a greedy optimization technique to compute the minimal perturbations that will maximize the model’s loss function during training, with respect to a surrogate model. This surrogate model is a linearized 2-layer graph convolutional network (GCN), which approximates how changes in the graph structure affect the classification result. The primary objective of NETTACK is to increase the model’s confidence in incorrect classifications by maximizing the distance between the logits of the perturbed graph and those of the clean graph. To achieve this, NETTACK identifies candidate edges for perturbation around a target node and evaluates a score function, for each connected edge. This score indicates how significantly altering a particular edge will impact the classification result of the target node. The edge with the highest score is then selected, and the graph is updated by either adding or removing that edge. Since NETTACK is a targeted attack (only cause misclassification of a specific single node), we adapt the same method in (Zügner and Günnemann 2019) to convert it into untargeted attack (compromise node classification performance of the model) which randomly select one test node and attack it using NETTACK while considering all nodes in the network. We then aggregate the result over the test nodes.

METAATTACK optimizes the graph structure directly via gradient descent by treating the adjacency matrix (graph structure) as a hyperparameter. It frames the attack as a bi-level optimization problem: the inner level minimizes the training loss on the original graph, while the outer level minimizes the loss on the poisoned graph. To achieve this, METAATTACK utilizes a surrogate model, specifically a linearized two-layer GCN, to approximate how changes in the graph structure affect model performance. A score function evaluates the impact of each perturbation, and the attack flips the sign of meta-gradients (gradient wrt hyperparameters) for connected nodes, indicating which edges to remove. The algorithm then greedily selects perturbations—either adding or removing edge based on those with the highest scores, to maximize the attack’s impact.

For both methods, we use the same unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which ensures that the graph’s degree distribution only changes slightly. Therefore, we only perturb 10% of the edges.

In our poisoning attack experiments with LLAGA, we employ the perturbed graph structure obtained from NETTACK or METAATTACK to generate the node-level sequences and retrain the projector that converts the node-level sequences into a sequence of token embeddings. Similarly, for GRAPH-PROMPTER, we utilize the perturbed graph structure, re-encode the graph using a graph attention network (GAT), and retrain the projector with the perturbed graph. For evasion attacks, the graph-aware LLM is trained on unperturbed data, with perturbations introduced only during the inference stage.

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:   $G' \leftarrow G$  ▷ Create copy of original graph
17:   $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:   $P \leftarrow \text{GetPlaceholders}(T)$ 
19:  if  $|P| > 0$  then
20:     $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:    if AttackStrategy = NI then
22:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:       $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:      for each placeholder position  $p \in P$  do
25:         $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:        Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:      end for
28:    else if AttackStrategy = SI then
29:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:       $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:      for first placeholder position  $p \in P$  do
32:        Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:        break ▷ Only use first placeholder
34:      end for
35:    else if AttackStrategy = MSI then
36:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:       $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:      for  $i = 1$  to  $|P|$  do
39:         $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:         $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:        Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:      end for
43:    end if
44:  end if
45:  return  $G'$ 
46: end procedure

```

A.4 Algorithm for Node Sequence Template Injection Attacks

We present the algorithm for the node sequence template injection attack in LLAGA in Algorithm 1. The model con-

structs a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14). When a node has fewer than k neighbors, placeholders are inserted, creating an attack surface. We exploit this by injecting malicious nodes into placeholder positions through graph manipulation. We propose three strategies: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* injects a high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). These uninformative nodes corrupt the node sequence, degrading the projection layer’s performance. The inclusion of uninformative nodes in the sequence corrupts the target’s representation, impairing training of the projection layer.

A.5 Ablation Studies

Here, we present the results for the ablation studies presented in the main paper.

Ablation Results: Varying Feature Perturbation In Figure 6, we present the performance of two graph-aware LLMs, LLAGA and GRAPHPROMPTER, as the percentage of perturbed features is varied. We observe that increasing the proportion of perturbed features consistently degrades model accuracy for both models, indicating their sensitivity to feature-level attacks.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 show that the choice of base LLM has minimal impact on both clean performance and adversarial vulnerability in the LLAGA framework. Across Cora, Citeseer, and PubMed, models built on Vicuna-7B and Llama2-7B achieve nearly identical accuracy under clean conditions and exhibit similar degradation under both poisoning and evasion attacks. For instance, under METAATTACK evasion, Cora performance drops to 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B. These differences are well within standard deviation. Similar patterns hold across all datasets and attack types. This indicates that attack effectiveness is largely independent of the base LLM.

A.6 Results on the ArXiv Dataset

We now evaluate our attack on the large-scale ArXiv dataset, which consists of 169,343 nodes, 1,166,243 edges, and has a sparsity of 0.81. This experiment aims to investigate the effectiveness of node sequence template attacks in large, real-world graphs. As shown in Table 9, we observe a significant gap between the success rates of poisoning and evasion attacks, with poisoning attacks consistently outperforming evasion across all three injection strategies, which is not the case for the other datasets, Cora, Citeseer and PubMed. This suggests that ArXiv may have inherent structural differences that make it more vulnerable to poisoning yet more robust to evasion. While the sparsity of the ArXiv dataset is very low, inserting a single supernode could significantly affect the model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples in the prompt, as shown in Appendix A.7, illustrate our adversarial attacks on real features from the Cora dataset. The “input” is the text received by the LLM where the attack is being applied. It is important to note that while a human reader perceives the original text and the perturbed text as visually identical or very similar, the LLM processes a sequence of different Unicode characters or a different character order, which can alter its behavior. We show this in Table 10 for homoglyph and Table 11 for reordering attack.

Feature Corrector Defense Template We designed a feature corrector module to mitigate the effects of imperceptible attacks, specifically homoglyph substitutions and character reorderings. This module is implemented as a prompt-based defense using LLMs. The prompt is carefully crafted to instruct the LLM to act as a linguistic validator, focusing on detecting and correcting subtle, human-imperceptible perturbations.

The template for our prompt is shown below. It explicitly states the task and provides a clear output format, which significantly improves the LLM’s performance.

A.8 Detailed Discussion

The findings of our work raise several critical insights into the vulnerabilities and design considerations of graph-aware LLMs. As the integration of graph-structured data into LLMs continues to gain traction for graph tasks, it is important to explore not only the performance benefits but also the vulnerabilities of these novel architectures. Our systematic investigation of adversarial attacks on two representative graph-aware LLMs, LLAGA and GRAPHPROMPTER —reveals significant weaknesses, particularly when exposed to poisoning and evasion attacks, which are traditionally used to undermine GNNs. Our key observations are discussed below.

Impact of Graph Encoding on Adversarial Vulnerability The key observation from our results is that the integration method of graph structure into LLMs can inadvertently increase their susceptibility to adversarial attacks. Specifically, the node sequence template employed in LLAGA appears to create a vulnerability that can be exploited by adversaries, leading to substantial degradation in model performance. This attack surface, unique to LLAGA, highlights a critical tradeoff in graph-aware LLM design: while the use of node-level templates provides an effective means of incorporating graph structure and achieves better performance on graph task, it also exposes the model to new attack vectors not present in traditional GNNs. On the other hand, GRAPHPROMPTER, which uses a GNN-based projector for graph encoding, demonstrates comparatively greater resilience to these attacks, underscoring the potential advantages of hybrid architectures that leverage the strengths of both LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations In addition to the vulnerabilities in the graph encoding techniques, our results also highlight the effectiveness of

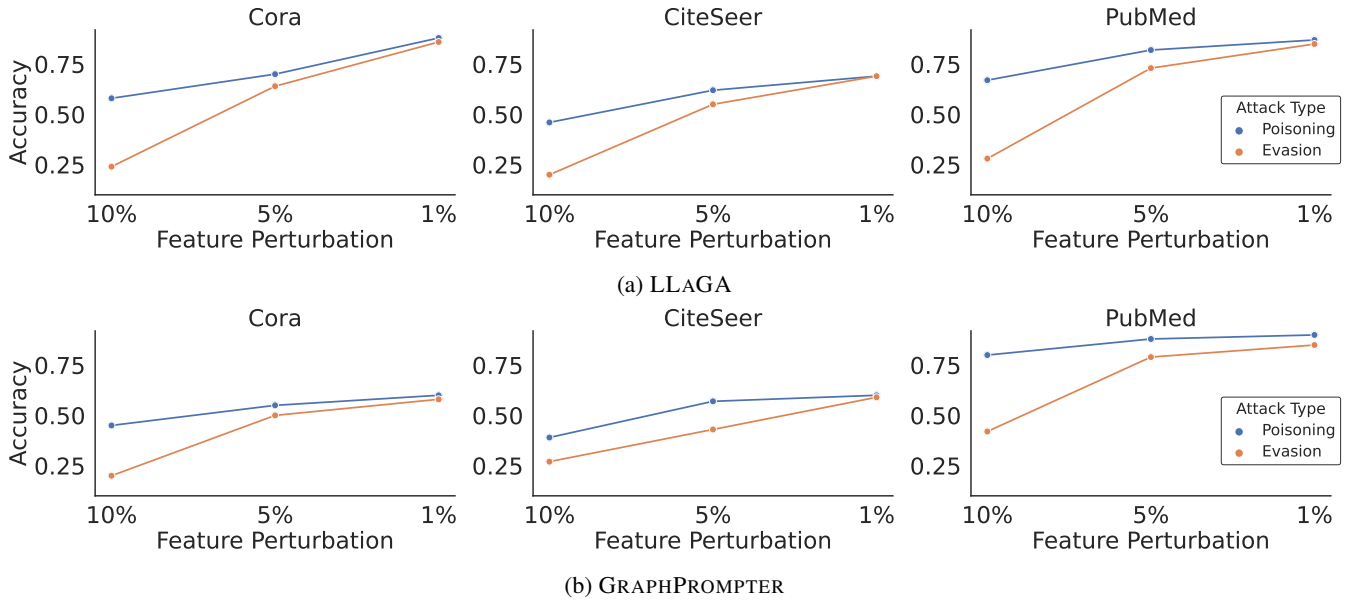


Figure 6: Performance of the Graph-aware LLM when the percentage of the perturbed feature is varied. The lower the accuracy, the better.

Table 8: Impact of base model changes on model’s performance and under different attacks. We show the result for LLAGA. The results demonstrate that the choice of base model does not significantly affect susceptibility to attacks.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

Table 9: Results of the node sequence template injection attack across different datasets and methods. *NI*, *SI*, and *MSI* indicates the non-adjacent, single supernode, and multiple supernodes injection attacks respectively. Clean refers to the performance of the model without any attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	—	0.73 ± 0.02	—
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

feature perturbation attacks across both graph-aware LLMs. Despite differences in how graph structure is encoded, both LLAGA and GRAPHPROMPTER exhibit significant sensitivity to imperceptible perturbations in the node features. This highlights a broader trend observed in the LLM paradigm: while these models excel in handling textual data, their robustness to adversarial perturbations, especially those targeting node features, remains a critical challenge. The ability of adversaries to degrade model performance by subtly altering node attributes without being detected calls into question the integrity of graph-aware LLMs, especially in security-sensitive applications. While text sanitization techniques, such as those discussed by (Boucher et al. 2022), offer potential defenses, addressing these imperceptible attacks such as homoglyph-based manipulations remains a complex challenge. As proposed, using LLMs as a corrector is one way to mitigate such vulnerabilities, by prompting the model to detect and repair suspicious or malformed inputs before downstream processing.

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly allcoct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: The prompt template used for the LLM-based feature corrector module. It explicitly defines the role, task, and expected output format for the LLM.

Table 10: An illustration of the imperceptible homoglyph attack. While the perturbed text is processed as distinct characters by the LLM, the visual output is nearly identical for a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtln squnc ptttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f stttisticl mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: An illustration of the imperceptible reordering attack. While the perturbed text is processed as distinct characters by the LLM, the visual output is nearly identical for a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings reveal that the efficacy of poisoning and evasion attacks in the context of graph-aware LLMs is not uniform across datasets. While attacks are particularly effective on the Cora, Citeseer and PubMed datasets, we observe a more nuanced interaction between the attack strategies and the graph structure on the ArXiv dataset. This variation emphasizes the importance of dataset characteristics in evaluating the vulnerabilities of graph-aware LLMs. As these models are increasingly deployed in diverse real-world settings, their vulnerabilities must be assessed not only in terms of algorithmic design but also in relation to the specific domains and data they are applied to.

Implications for Graph-aware LLM Development The vulnerabilities exposed in this study have broader implica-

tions for the future development and deployment of graph-aware LLMs. Given their increasing adoption for tasks that were traditionally the domain of GNNs, it is crucial to consider the tradeoffs between performance and vulnerabilities in the design of these paradigms. As graph-aware LLMs continue to evolve, there is a pressing need for the development of more robust encoding techniques and defenses against adversarial manipulation, particularly in the context of feature perturbations. Furthermore, understanding how adversarial attacks transfer from GNNs to graph-aware LLMs opens new avenues for research in protecting these models against sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs Defenses on adversarial attacks based on feature perturbation. Adversarial attacks on GNNs have predominantly

focused on structural perturbations, as feature-based perturbations typically fail to influence the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019), making them ineffective attack strategies. Consequently, there are *no known defenses against feature-based attacks on GNNs*, given their limited effectiveness. However, as demonstrated in the main paper, attacks based on feature perturbations proves effective in the graph-aware LLM paradigm. This introduces a novel challenge within the graph-aware LLM framework, where feature-based attacks, previously ineffective against GNNs, can now successfully manipulate model behavior. This shift highlights the need for developing new defenses that address both structural and feature-based adversarial perturbations. While text sanitization techniques, such as the use of optical character recognition (OCR) as a pre-processing step for encoding text for the LLM, offer potential defenses, addressing these imperceptible attacks such as homoglyph-based manipulations remains a complex challenge as OCR is imperfect and tends to misinterpret homoglyph at a higher rate than unperturbed text (Boucher et al. 2022). This led to our design of self-correcting mechanism for feature perturbation using LLMs.

Defenses on adversarial attacks based on structural perturbation. Due to the transferability of structure-based adversarial attacks from GNNs to graph-aware LLMs, particularly in evasion attacks, it is natural to expect that defenses developed for GNNs would also be effective in this paradigm. However, this assumption does not hold. Unlike traditional GNN settings, *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective.*

One such defense mechanism in GNNs involves making the adjacency matrix trainable, allowing edge weights to adapt during training to mitigate adversarial perturbations. This approach assigns lower weights to edges connecting dissimilar nodes and higher weights to edges linking nodes with greater feature similarity. Several **preprocessing-based** defenses have been proposed building on this approach, such as removing edges between nodes with low feature similarity (Wu et al. 2019) or introducing randomized edge dropping (Dai et al. 2018). However, these defenses rely heavily on clean, unperturbed node features to compute reliable similarity scores. When node features are also perturbed, as in the case of our attacks, such defenses become ineffective and may even exacerbate the problem by connecting incorrect edges. We demonstrate this effect on a random node in the Cora dataset (Figure 8). As observed, while the clean feature similarity graph preserves some original edges alongside introducing some wrong connections, the edges reconstructed by the perturbed features fail to preserve any original connections, leading to entirely incorrect reconstructions.

Another line of work focuses on decoupling reliance on the graph structure. For instance, (Shanthamallu, Thiagarajan, and Spanias 2021) proposed a defense that trains a surrogate predictor dependent solely on node features and employs an uncertainty matching strategy to extract graph information from the GNN. Similarly, Wu et al. (2021) introduced a

co-training framework that trains submodels independently on feature and structural views, allowing knowledge distillation through the inclusion of confident unlabeled data into the training set. However, these approaches also assume access to unperturbed node features, an assumption that does not hold in our setting, where both structure and features are adversarially perturbed. Consequently, structural defense strategies developed for GNNs are not directly applicable to the graph-aware LLM paradigm, which calls for novel defenses tailored to this paradigm.

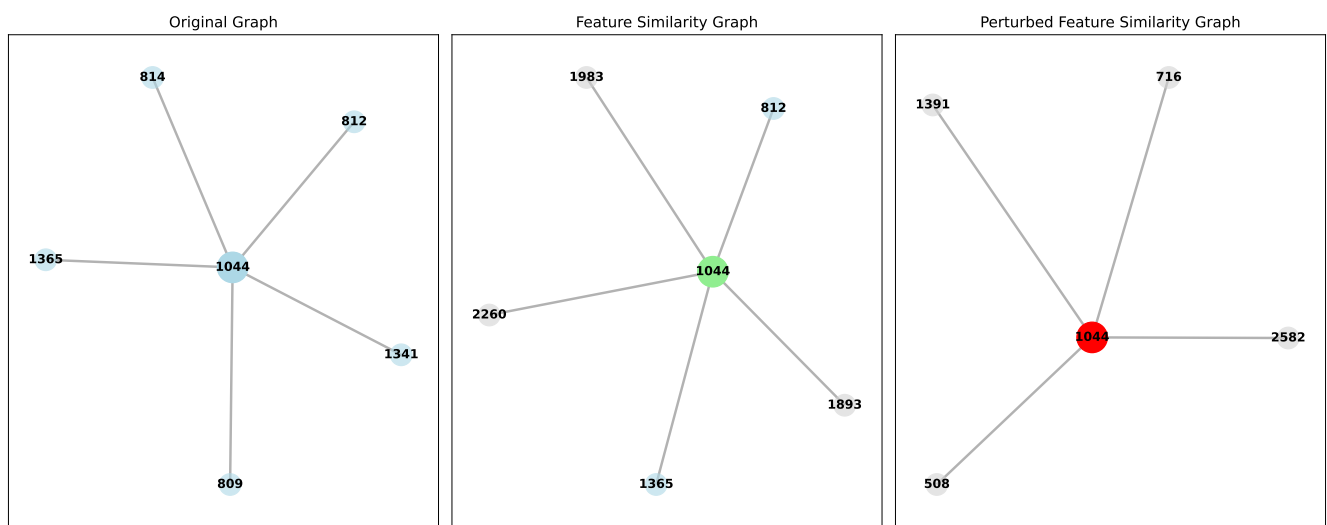


Figure 8: Edges reconstructed for a random node 1044. **Original (left)** is the ground truth edges from the original graph. **Feature Similarity (middle)** shows the edges reconstructed using clean node features, where blue nodes indicate preserved connections from the original graph. **Perturbed Feature Similarity (right)** shows the edges reconstructed using perturbed node features.